

Security Audit — k3s home-cluster

A practical, risk-ranked review of the live cluster and the public GitOps repo. Scoped to a solo home-lab threat model — accidental exposure, a leaked secret in a public repo, one compromised service pivoting across a flat network, and data loss — not an enterprise checklist.

Date · 2026-06-30

Cluster · 4 nodes · k3s v1.35.5

Method · live kubectl + full git-history scan

Repo · PUBLIC

Scorecard

1

CRITICAL

No-login cluster-admin on the tailnet

2

HIGH

Open registry · open vault signups

8

MEDIUM

Network, secrets, images, exposure

4

LOW / EXPECTED

Justified GPU/monitoring access

Overall posture — **reasonable, with a few sharp edges**

Repo hygiene is genuinely good: **no hard secrets are committed** in the current tree or full history — your `.gitignore` discipline works. The risk is concentrated in **exposure** (a couple of powerful, unauthenticated services reachable beyond where they should be) and **architecture** (one unencrypted SQLite control plane holding every secret; a flat pod network with zero policies). Everything below is cheap to fix at your stage.

Fix these four this week

- 1

Headlamp

— take it off the tailnet & set role to view

C1

~5 min
- 2

Vaultwarden

— SIGNUPS_ALLOWED:"false"

H2

~2 min
- 3

Registry

— add htpasswd basic-auth

H1

~30 min
- 4

Docs

— scrub tailnet 100.x IPs from docs/07

M5

~5 min

Critical

CRITICAL C1 Headlamp runs as no-login cluster-admin, exposed over Tailscale

WHAT Headlamp authenticates every visitor as a pod ServiceAccount bound to `cluster-admin`, with **no login prompt**. The values file is commented "LAN-only" — but the Tailscale Ingress right beside it publishes it at `headlamp.<tailnet>.ts.net`. Confirmed live: binding `headlamp-admin` → `cluster-admin`.

WHY Anyone who reaches that tailnet host — any tailnet member, any device that ever receives a tagged key, or one ACL slip — gets **full read/write control of the entire cluster**, no password. Highest-impact finding: game-over if reached.

WHERE `clusters/home/headlamp/values.yaml` `clusters/home/tailscale/ingress-headlamp.yaml`

FIX Remove `ingress-headlamp.yaml` from the tailscale kustomization (keep it `.lan-only`), **and** set `clusterRoleName: view` + disable `unsafeUseServiceAccountToken`. Recommended: do both. **~5 min**

High

HIGH H1 Private registry: no auth, plaintext, push & pull open on the LAN

WHY NodePort 30500, no `htpasswd`, `insecure_skip_verify`, reachable on every node IP. Anyone who reaches a node can **push** an image — a poisoned image is direct code execution on the cluster (every node pulls from it).

WHERE `clusters/home/registry/registry.yaml`

FIX Add an `htpasswd` Secret (basic auth); stop exposing it past where it's needed — pushes should come from CI. **~30 min**

HIGH H2 Vaultwarden allows open signups and is on the tailnet

WHY This is the app that stores **every other credential**. `SIGNUPS_ALLOWED: "true"` + `SIGNUPS_VERIFY: "false"` means anyone who reaches it can create an account on your vault.

WHERE `clusters/home/vaultwarden/vaultwarden.yaml:79`
`clusters/home/tailscale/ingress-vaultwarden.yaml`

FIX Set `SIGNUPS_ALLOWED: "false"` now that your account exists; flip back on only transiently to add a user. **~2 min**

MED M1 **No NetworkPolicies**

0 policies across 28 namespaces — any pod can reach any service. One compromised `:latest` image can hit Vaultwarden's DB, the registry, MinIO, Postgres.

Fix: default-deny + allow on the 2–3 crown jewels (vaultwarden, registry, platform Postgres). No mesh needed.

MED M2 **Secrets unencrypted at rest**

a3 is the sole control plane on **SQLite**, secrets base64-only. Whoever gets a3's disk or a raw backup gets **every** secret — and it's a single API SPOF.

Fix: `k3s secrets-encrypt enable`; pair with the planned HA/etcd migration.

MED M3 **No managed secrets**

No Sealed/External Secrets/SOPS. Secrets live only in live SQLite + your head — the GitOps repo isn't actually reproducible.

Fix: **Sealed Secrets** — encrypted secrets become safe to commit. Great "next infra" project.

MED M4 **:latest images**

16 of 70 images float on `:latest` (invokeai is untagged). Non-reproducible; a re-pull can change or poison the image; no rollback.

Fix: pin a tag/digest per image; internet-facing first.

MED M5 **Tailnet IPs in public repo**

`docs/07:19-22` lists real `100.x` tailnet IPs (→ VNC) — the repo's own stated red line. Not exploitable (CGNAT) but should be scrubbed.

Fix: replace with node names / `<tailnet>`.

MED M6 **No Pod Security baseline**

No namespace carries a `pod-security.kubernetes.io` label — nothing stops a future manifest running privileged/hostPath by accident.

Fix: **enforce: baseline** on app namespaces; leave GPU/monitoring at **privileged**.

MED M7 **NodePort sprawl (14)**

14 services on every node IP — incl. unauthenticated **prometheus**, **minio** console, and **litellm** (master key). Gated only by "the LAN is trusted."

Fix: prefer `.lan`/Tailscale Ingress; drop the NodePort once an Ingress exists (Phoenix is the template).

MED M8 **Root containers**

~10 of ~44 workloads set any `securityContext`; a few are explicitly `runAsUser:0`. A container escape starts as root.

Fix: add `runAsNonRoot` + `seccompProfile: RuntimeDefault` on new services. Low priority on an isolated LAN.

Low / Expected — noted, not alarming

ID	FINDING	WHY IT'S OK (FOR NOW)
L1	Privileged & host-level DaemonSets (gpu-feature-discovery, dcgm, node-exporter, vram-reporter, nvidia-device-plugin, promtail, nfd, netdata-child)	All justified by GPU/node telemetry. Keep them scoped to their system namespaces — they're the highest-value escape targets, so don't add more.
L2	Traefik dashboard enabled (<code>--api.dashboard=true</code>)	Not routed externally — no IngressRoute exposes it. Just don't add one.
L3	No backups	Integrity/availability, not confidentiality. Covered by the storage+HA plan; k8up → MinIO is the cheap first step.
L4	Helm charts pinned but unverified	Version-pinned from official repos into a gitignored <code>charts/</code> . No digest provenance, but reasonable here.

What to do next (after the four quick wins)

These line up with your existing roadmap — each is a real "stand up a piece of infra and understand it" project, ordered by value.

- **Sealed Secrets** **M3** — makes the GitOps repo truly reproducible. Best next project: install the controller, convert existing Secrets to `SealedSecret` CRDs.
- **secrets-encryption + HA/etcd** **M2** — enable encryption at rest on a3, then fold the etcd/HA migration into the storage+HA plan.
- **A few NetworkPolicies** **M1** — default-deny on vaultwarden / registry / platform first; expand from there.
- **Pin :latest images** **M4** — a few per session, internet-facing first.
- **k8up backups → MinIO** — the reliability win from the earlier project menu; protects Paperless, Grafana, Phoenix, Forgejo, Vaultwarden.

Bottom line. Knock out the four quick wins this week (issues are filed in `brian/home-lab`), then treat Sealed Secrets + secrets-encryption + a handful of NetworkPolicies as your next infra projects. You're in good shape for a home lab — these changes move you from "trusts the LAN" to "defends in depth."